

CHAPTER 08 텍스트 분석

NLP와 텍스트 분석이란?

NLP(Natural Language Processing)와 텍스트 분석(Text Analytics, 이하 TA)은 구분하여 이해해야 합니다.

- NLP: 미신이 인간의 언어를 이해하고 처리하는 기반 기술
- 텍스트 분석: 비정형 텍스트에서 의미 있는 정보를 추출하는 기법 → 텍스트 마이닝(Text Mining)으로도 불림

주요 텍스트 분석 기법

- 텍스트 분류(Text Classification): 문서를 특정 범주로 분류
- 감성 분석(Sentiment Analysis): 텍스트에서 긍정/부정 감정을 분석
- 텍스트 요약(Summarization): 중요한 정보를 요약 → 토픽 모델링(Topic Modeling)
- 텍스트 군집화(Clustering)와 유사도 측정: 유사한 문서들을 그룹화

텍스트 분석 프로세스

전체 프로세스 흐름

Text 문서 → Feature Vectorization (Bag of Words / Word2Vec) → ML 학습/예

텍스트 분석은 3단계로 진행됩니다:

1. 텍스트 전처리 → 클린징, 토큰화, 정규화
 2. 피쳐 벡터화 → BOW, Word2Vec 등으로 수치화
 3. ML 모델 적용 → 학습 및 예측/평가
-

텍스트 전처리

정규화 단계

- 필터링/스톱 단어 제거: 불필요한 단어 제거
- Stemming: 단어의 어근 추출 (working → work)
- Lemmatization: 단어의 기본형으로 변환 (는 → 다)

문장 토큰화 (Sentence Tokenization)

문장의 마침표(.), 개행(\n) 등을 기준으로 문장을 분리합니다. NLTK의 `sent_tokenize()` 함수를 사용합니다.

```
from nltk import sent_tokenize
import nltk
nltk.download('punkt')

text_sample = "The Matrix is everywhere... You can see it when..."
sentences = sent_tokenize(text)
```

단어 토큰화 (Word Tokenization)

문장을 개별 단어로 분리하는 과정입니다. NLTK에서는 여러 API 제공:

- `word_tokenize()`: 기본 분리
- `RegexTokenizer`: 정규표현식 기반 분리

Stemming과 Lemmatization

Stemming (어근 추출)

Lancaster Stemmer를 사용하여 단어의 어근을 추출합니다.

```
from nltk.stem import LancasterStemmer
stemmer = LancasterStemmer()
```

```
print(stemmer.stem('working')) # work
print(stemmer.stem('amusing')) # amus
print(stemmer.stem('happier')) # happy
print(stemmer.stem('fancier')) # fanciest
```

특징: 단어의 어미를 제거하여 기본형으로 변환하되, 완벽한 단어를 만들지 않음

Lemmatization (기본형 변환)

WordNetLemmatizer를 사용하여 단어의 정확한 기본형을 찾습니다.

```
from nltk.stem import WordNetLemmatizer
import nltk
nltk.download('wordnet')

lemma = WordNetLemmatizer()
print(lemma.lemmatize('amusing', 'v')) # amuse
print(lemma.lemmatize('happier', 'a')) # happy
print(lemma.lemmatize('fancier', 'a')) # fancy
```

특징: 품사 정보를 활용하여 정확한 기본형으로 변환

Bag of Words (BOW)

BOW의 개념

단어들을 단순히 출현 횟수로 표현하는 방식입니다. 문서의 단어들이 무엇이고 얼마나 등장했는지에만 집중합니다.

문제점: - 단어의 의미 관계를 고려하지 않음 - 희소 행렬(Sparse Matrix) 생성 → 메모리 낭비
- 단어의 순서를 무시함

BOW 구현 예시

	and	baseball	daughter	games	likes	my	play	to	too	watch	wife
문 서 1	2	2	1	2	2	2	0	2	1	2	2
문 서 2	1	1	0	0	1	1	1	1	0	1	1

피처 벡터화 방법

CountVectorizer (BOW 기반)

BOW 방식으로 텍스트를 수치화합니다. 각 단어의 출현 빈도를 행렬로 표현합니다.

주요 파라미터: - max_df: 상위 몇 %의 단어 제외 - min_df: 하위 몇 %의 단어 제외 - max_features: 최대 피처 개수 - stop_words: 제거할 단어 목록 - n_gram_range: n-gram 범위 설정 (1, 2)는 단어와 2-gram 모두 사용

```
from sklearn.feature_extraction.text import CountVectorizer

vect = CountVectorizer(stop_words='english', ngram_range=(1, 2))
x_train = vect.fit_transform(train_data)
x_test = vect.transform(test_data)
```

TF-IDF Vectorizer

단어의 중요도를 반영하여 가중치를 부여합니다.

- TF (Term Frequency): 문서 내 단어 빈도
- IDF (Inverse Document Frequency): 전체 문서에서 단어의 희귀성

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(stop_words='english', ngram_range=(1, 2))
x_train = tfidf.fit_transform(train_data)
x_test = tfidf.transform(test_data)
```

Pipeline을 이용한 텍스트 처리

Pipeline은 여러 처리 단계를 자동으로 연결하여 코드를 간결하게 합니다.

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline([
    ('tfidf_vect', TfidfVectorizer(stop_words='english', ngram_range=(1, 2))),
    ('lr_clf', LogisticRegression(solver='liblinear', C=10))
])

pipeline.fit(X_train['review'], y_train)
pred = pipeline.predict(X_test['review'])
```

CHAPTER 05 감성 분석

감성 분석이란?

감성 분석(Sentiment Analysis)은 문서의 주관적인 감정/의견(긍정/부정/중립 등)을 분석하는 기법입니다.

감성 분석의 방식

1. 지도학습: 학습된 데이터 기반으로 레이블 예측
2. 비지도학습: Lexicon 기반으로 감성 판정

Lexicon 기반 감성 분석

Lexicon은 감성 어휘사전으로, 각 단어가 긍정/부정의 감정을 가지는지 정의합니다.

장점: 명시적 규칙 기반으로 해석 가능
단점: 문맥을 고려하기 어려움

지도학습 기반 감성 분석 - IMDB 영화평

데이터 소개

IMDB의 영화 리뷰 데이터를 활용합니다. Kaggle에서 "Bag of Words Meets Bags of Popcorn" 경진대회 데이터를 사용합니다.

- **데이터:** labeledTrainData.tsv (11314개 영화 리뷰)
- **컬럼:** id, sentiment (1: 긍정, 0: 부정), review (리뷰 텍스트)

```
import pandas as pd

review_df = pd.read_csv('./labeledTrainData.tsv', header=0, sep="\t", q
review_df.head(3)
```

id	sentiment	review
5814_8	1	With all this stuff going down...
2381_9	1	"The Classic War of the Worlds"...
7759_3	0	The film starts with a manager...

데이터 설명

- id: 각 데이터의 고유 id

- sentiment: 1 = 긍정 평가, 0 = 부정 평가
- review: 영화평 텍스트

텍스트가 너무 길면 텍스트 길이가 단어를 더 지배하게 되므로 주의 필요합니다.

CountVectorizer 기반 감성 분석

피처 벡터화 시 TF-IDF 벡터화를 적용하여 예측 성능을 높입니다.

```
pipeline = Pipeline([
    ('tfidf_vect', TfidfVectorizer(stop_words='english', ngram_range=(1, 2)),
    ('lr_clf', LogisticRegression(solver='liblinear', C=10))
])

pipeline.fit(X_train['review'], y_train)
pred = pipeline.predict(X_test['review'])

print('예측 정확도는 {0:.4f}, ROC-AUC는 {1:.4f}'.format(
    accuracy_score(y_test, pred),
    roc_auc_score(y_test, pred_probs)))
```

출력: 예측 정확도는 0.8936, ROC-AUC는 0.9598

TF-IDF 기반 피처 벡터화의 예측 성능이 훨씬 나아졌습니다.

비지도학습 기반 감성 분석 - Lexicon 활용

Lexicon의 개념

Lexicon은 감성 어휘사전으로, 각 단어의 감성 극성(polarity score)을 정의합니다.

극성(Polarity): - 긍정(Positive): 1에 가까운 값 - 부정(Negative): -1에 가까운 값 - 중립: 0에 가까운 값

NLTK의 감성 분석

NLTK는 다양한 감성 분석 도구를 제공합니다:

- TextBlob: 간단한 감성 분석
 - VADER: 소셜 미디어 텍스트용 감성 분석
 - SentiWordNet: 워드넷 기반 감성 사전
-

실습 순서

1. 데이터 로드 → labeledTrainData.tsv 로드
2. 텍스트 전처리 → 토큰화, 정규화
3. 피처 벡터화 → CountVectorizer 또는 TfidfVectorizer
4. 모델 학습 → Logistic Regression 또는 다른 분류기
5. 예측 및 평가 → Accuracy, ROC-AUC 등 측정